1. Suppose you have a distributed application which runs on 10 computer nodes. The computer nodes are spread across several floors of the same building. The distributed application is multi-process application. The multiple processes which are part of this application need to exchange data to jointly perform some tasks. However, these processes bear no parent-child relationships. The communication between each pair of communicating processes may be unidirectional or bidirectional. None of the communicating processes will be involved in bulk data transfer. You are ready to tolerate latency in the data transfer process and will not be implementing any synchronization mechanism. Every process mandatorily needs to explicitly name the sender when it acts as the receiver or name the receiver when it acts as the sender. No mailboxes are to be used. Which of the following would you use to facilitate this distributed IPC? 1 point

- ordinary pipe
- shared memory
- indirect message passing
- (●) direct message passing

2. Suppose you want to implement a multithreaded application containing 15 threads of control on a multicore system. Consider these threads to be user level threads. You do not want the entire application to block even if one of the user level threads makes a blocking system call. You also do not want to create more than 13 kernel level threads. Moreover, for 5 out of the 15 user level threads, you want to bind each user level thread to a single kernel level thread. Which multithreading model is most appropriate for your application? 1 point

- Many-to-Many model
- Many-to-One model
- (●) Two-level model
- One-to-One model

3. Consider the following program. How many times will GOOD be printed on the console on executing this program on an Ubuntu operating system? Assume all API calls are executed successfully. 1 point

```
#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

int main() {

fork();

fork();

fork();

fork();

fork();

printf("\nGOOD\n");

return 0;

}
```

- (●) 32 times
- ( ) 64 times
- ( ) 6 times
- ( ) 5 times

4. Suppose you have a large sized dataset on which you must perform 5 operations, namely, calculating sum, calculating median, calculating mode, finding the maximum value and finding the minimum value. You have created 6 separate threads, T1, T2, T3, T4, T5 and T6. T1 and T2 perform the sum operation with T1 working on half of the dataset and T2 working on the remaining half of the dataset. T3 calculates the median, T4 calculates the mode, T5 calculates the maximum and T6 calculates the minimum. This is an example

1 point

- (●) Both data and task parallelism
- ( ) Data parallelism
- ( ) Task parallelism
- ( ) No parallelism at all

5. Which of the following is not shared by threads belonging to the same process?

1 point

- ( ) data
- (●) stack
- ( ) files
- ( ) code

6. What is the output of the following C code snippet on an Ubuntu system? Note that the entire program has not been shown. But you have enough information to figure out the output. Assume all relevant header files have been included, the program compiles successfully and all function calls execute successfully. 1 point

```
pthread_t tid1, tid2;

pthread_attr_t attr1, attr2;

void *arg;

pthread_attr_init(&attr1);

pthread_attr_init(&attr2);

pthread_create(&tid1, &attr1, fun, arg); //fun() is to be executed by new thread

pthread_create(&tid2, &attr2, fun, arg); //fun() is to be executed by new thread

printf("\n%d", pthread_equal(tid1, tid2));
```

- ○ 3
- ○ 2
- ○ 1
- ◉ 0

7. Which of the following is not a valid process state transition? 1 point

- ○ ready to running
- ○ new to ready
- ◉ new to waiting
- ○ running to waiting

8. Consider the following program. What is the output of executing this program on an Ubuntu operating system? Assume all API calls are executed successfully. 1 point

```
#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

int main() {

pid_t p1, p2;

p1 = fork();

if(p1 == 0){

p2 = fork();

if(p2 == 0){

printf("\nHELLO\n");

execlp("/bin/eco", "eco", "PRINT", NULL); printf("\nDONE\n");

}

else {

wait(NULL);

printf("\nFINISH\n");

}

}

else {

wait(NULL);

execlp("/bin/echo", "echo", "OS", NULL);

printf("\nEXIT\n");

}

return 0;

}
```

- ○ First HELLO is printed, then PRINT is printed, then FINISH is printed and finally, OS is printed.
- ◉ First HELLO is printed, then DONE is printed, then FINISH is printed and finally, OS is printed.
- ○ First HELLO is printed, then PRINT is printed, then DONE is printed, then FINISH is printed, then OS is printed and finally, EXIT is printed.
- ○ First HELLO is printed, then DONE is printed, then FINISH is printed, then OS is printed and finally, EXIT is printed.

9. In a TCB, which of the following components is crucial for saving the thread's execution context during a context switch? 1 point

- ○ The code section of the thread.
- ◉ The thread's execution context, including register values, program counter, and stack pointer.
- ○ The thread's file descriptors and network connections.
- ○ The thread's memory management information, including page tables and virtual memory mappings.

10. When a process P is selected by the short-term scheduler, which one of the following state transitions occurs for P? Note that you do not have to worry about the state transition of any process other than P. 1 point

- ◉ ready to running
- ○ running to waiting
- ○ running to terminated
- ○ waiting to ready